Exercises 4.1

1) Keep the base case as is. In the recursive case, first check if n is even or odd. If n is even, then continue as is. If n is odd, we add a column of 0s to the ~~bott~~right end and a row of 0s to the bottom of A, B and C to get A', B' and C'. Now we simply split them each ~~ito~~ into ~~n/~~ $n+1/2 \times n+1/2$ submatrices, and make the same recursive calls ~~as~~ as before (except with $n = n+1$). ~~Once~~ The resulting matrix, if n ~~were~~ was odd, is an ~~n×n~~ $n+1 \times n+1$, with the final row and column being all 0s. We remove that column and row to get our result i.e $C + A \cdot B$.

We can pad 0s conceptually ~~rat~~ using indices rather than copying the matrices into new $n+1 \times n+1$ arrays and thus get $\Theta(1)$ time, ~~same~~ ~~fo~~ since it is just a single row and column of 0s; ~~thus getting $\Theta(1)$ time for both the padding~~ a. We similarly get $\Theta(1)$ time for removing the padding as well. We already know splitting the matrices take $\Theta(1)$ time as it is just arithmetic on indices. We have 8 recursive calls, with $(n+1)/2$ as the 4th argument if n is odd, and $n/2$ otherwise. Thus we get the recurrence:-

$T(n) = 8T(\lceil n/2 \rceil) + \Theta(1)$

In the worst case, every n in every recursive call is odd, so $\lceil n/2 \rceil = n+1/2$ every time. Each recursive call produces 8 subproblems of size $n+1/2$, thus the recursion bottoms out after roughly $\lg(n+1)$ calls. Thus we have $8^{\lg(n+1)}$ 'leaves' on the tree where each does $\Theta(1)$ extra work. $8^{\lg(n+1)} = 2^{3\lg(n+1)}$
$= 2^{\lg(n+1)^3} = (n+1)^3 = n^3 + 3n^2 + 3n + 1$
$= \Theta(n^3)$.

2) Let A be the $kn \times n$ matrix and B the $n \times kn$ matrix. We divide A into k '$n \times n$' submatrices stacked vertically and B into k '$n \times n$' matrices stacked horizontally. The submatrices are labelled $A_1 \cdots A_k$ from top to bottom and $B_1 \cdots B_k$ from left to right.

Initialise a $kn \times kn$ array of 0s to represent C. So we have k^2 '$n \times n$' submatrices that are labelled C_{ij}, where j is its position horizontally from left to right and i its position vertically from top to bottom.

~~Each~~ Each 'C_{ij}' is the result of $A_i \times B_j$. We have k^2 blocks, and each $A_i \times B_j$ is handled by

the generalised version of MATRIX-MULTIPLY-RECURSIVE described in the answer to the previous question, which takes time $\Theta(n^3)$. Also, initialising an array for C takes $\Theta(k^2n^2)$ time, so our runtime is $\Theta(k^2n^3) + \Theta(k^2n^2) + \Theta(1)$ (the $\Theta(1)$ is for dividing the arrays using index manipulation) which evaluates to $\Theta(k^2n^3)$

For multiplying an $n \times kn$ matrix with a $kn \times n$ matrix instead, (i.e. $B \cdot A$ instead of $A \cdot B$) we do the same division into submatrice of size $n \times n$ as before, except for C we initialise an $n \times n$ matrix of 0s instead. To get the result of $B \cdot A$ in C, we do

$C = \cancel{A_1B_1 + A_1B_1 + A_2B_2 + \dots + A_kB_k}$

$B_1A_1 + B_2A_2 + \dots + B_kA_k$.

Each product is one function call and we have n products, as well as the time for initialising an $n \times n$ array for C, and dividing A & B into submatrices, thus getting $\Theta(kn^3) + \Theta(n^2) + \Theta(1) = \Theta(kn^3)$.

3) Before the recursive calls, for each matrix A, B and C, we have to initialise 4 $n/2 \times n/2$ arrays and copy a submatrix of size $n/2$ into each of them, thus doing work that takes time $12(n/2)^2 = 3n^2 = \Theta(n^2)$. Also, after the recursive calls we have to copy 4 $n/2 \times n/2$ matrices (representing C_{11}, C_{12}, C_{21} and C_{22}) into 1 $n \times n$ matrix to represent C, which takes time $n^2 = \Theta(n^2)$. Thus our total non recursive work is $2\Theta(n^2) = \Theta(n^2)$ and so we get the recurrence:-

$$T(n) = 8T(n/2) + \Theta(n^2).$$

By definition, $\forall f(n) \in \Theta(n^2)$, $f(n) \in O(n^2)$. Also, $n^2 = n^{\lg 8 - 1}$ so $f(n) = O(n^{\lg 8 - 1})$ so the master theorem case 1 applies and we have $T(n) = \Theta(n^{\lg 8}) = \Theta(n^3)$.

4) Assume n is a power of 2, and C is any matrix of the same size as A & B (unlike the multiply procedure C does not have to be a zero matrix to hold A+B):-

```
MATRIX-ADD-RECURSIVE(A,B,C,n)

if n==1
    C11 = A11 + B11
    return
```

Partition A, B and C into $n/2 \times n/2$ submatrices $A_{11}, A_{12}, A_{21}, A_{22}$; $B_{11}, B_{12}, B_{21}, B_{22}$; and $C_{11}, C_{12}, C_{21}, C_{22}$; respectively

MATRIX-ADD-RECURSIVE $(A_{11}, B_{11}, C_{11}, n/2)$
MATRIX-ADD-RECURSIVE $(A_{12}, B_{12}, C_{12}, n/2)$
MATRIX-ADD-RECURSIVE $(A_{21}, B_{21}, C_{21}, n/2)$
MATRIX-ADD-RECURSIVE $(A_{22}, B_{22}, C_{22}, n/2)$.

~~Ind~~ Partitioning through indexing takes time $\Theta(1)$, and there are 4 recursive calls, so our running time is:-

$$T(n) = 4T(n/2) + \Theta(1)$$

By definition of Θ, $\forall f(n) \in \Theta(1)$, $f(n) = O(1) = O(n)$ and $n^{\lg 4 - 1} = n$ thus ~~$T(n)$~~ $f(n) = O(n^{\lg 4 - 1})$ and therefore by Master Theorem case 1, $T(n) = \Theta(n^{\lg 4}) = \Theta(n^2)$

If we use $\Theta(n^2)$ time copying instead of index calculations for partitioning, we would have running time:-

$$T(n) = 4T(n/2) + \Theta(n^2)$$

Since $n^{\lg 4} \lg^0 n = n^2$, ~~by~~ and by definition, $\forall f(n) \in \Theta(n^2)$, $f(n) = \Theta(n^2) = \Theta(n^{\lg 4} \lg^0 n)$, case 2 of the master theorem applies and thus we have $T(n) = \Theta(n^{\lg 4} \lg n)$
$= \Theta(n^2 \lg n)$